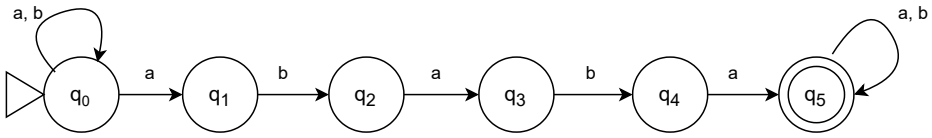
[redacted]
Student ID: [redacted]
Assignment: 1

1a. L1 = {w ∈ {a, b}∗| w = uvuv for some u, v ∈ {a, b}∗}
For L1 to be regular w = uvuv for some u,v = {a,b}* must have an applicable NDFSM. However, because u and v can come from {a,b}*, this means that they can be any combination of a and/or b or ε, meaning there would be no set way to define L1 in a FSM. By the pumping theorem if L1 is regular, then there exists a k >= 1 such that any w in L1 with |w| >=k can be written as w = xyz for some x, y, z in Σ* where $\forall$q >= 0, xy^qz is in L1. If y were selected as a section in the first k letters of w, and was pumped, any uvuv for u, v = {a,b}* would no longer hold. Because L1 fails the pumping theorem, L1 is considered not regular

1b. L2 = {w ∈ {a, b}∗| w = uvu for some u, v ∈ {a, b}∗}
For L2 to be regular w = uvu for some u,v = {a,b}* must have an applicable NDFSM. However, because u and v can come from {a,b}*, this means that they can be any combination of a and/or b or ε meaning there would be no set way to define L2 in a FSM. By the pumping theorem if L2 is regular, then there exists a k >= 1 such that any w in L2 with |w| >=k can be written as w = xyz for some x, y, z in Σ* where $\forall$q >= 0, xy^qz is in L2. If y were selected as a section in the first k letters of w, and was pumped, any uvu for u, v = {a,b}* would no longer hold. Because L2 fails the pumping theorem, L2 is considered not regular

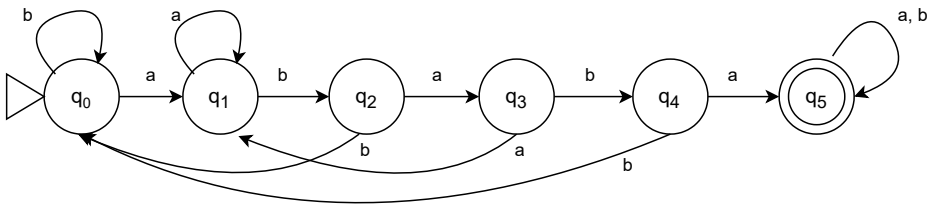
1c. L3 = {w ∈ {a, b}∗ | w has ababa as a substring}

NDFSM to test if L3 is regular



By the pumping theorem, if L3 is regular, then there exists a k >= 1 such that any w in L3 with |w| >=k can be written as w = xyz for some x, y, z in Σ* where $\forall$q >= 0, xy^qz is in L3. For L3, there are 6 states (k) and if y was selected as the first or last state to be repeated, the result would still be an element of L3, with "ababa" as a substring. Because L3 passes the pumping theorem and a proper non-deterministic state automata can be formed to accept L3, L3 is regular.

DFSM



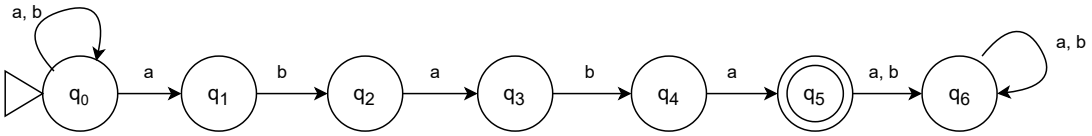
q	a	b
0	1	0
1	1	2
2	3	0
3	1	4
4	5	0
5	5	5

The DFSM for L3 has no states that are equivalent as seen in the path chart on the right so the DFSM is also its minimized DFSM.

Any number of a's and b's can be at the beginning and/or end of a word that is accepted by L3 while it has to have 'ababa' as a substring so the regular expression for L3 is: a*b*ababaa*b*

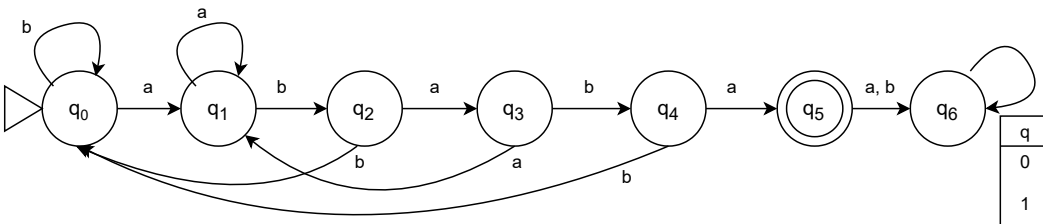
1d. L4 = {w ∈ {a, b}∗| w has ababa as a suffix}

NDFSM to test if L4 is regular



By the pumping theorem, if L4 is regular, then there exists a k >= 1 such that any w in L4 with |w| >=k can be written as w = xyz for some x, y, z in Σ* where $\forall q >= 0$, xy^qz is in L4. For L4, there are 7 states (k) and if y was selected as the first state to be repeated, the result would still be an element of L4, with "ababa" as a suffix. Because L4 passes the pumping theorem and a proper non-deterministic finite automata can be formed to accept L4, L4 is regular.

DFSM

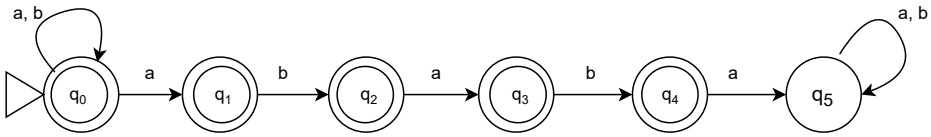


q	a	b
0	1	0
1	1	2
2	3	0
3	1	4
4	5	0
5	6	6

Excluding the fact that L4 has state 6 as a dead zone, the DFSM for L4 has no states that are equivalent as seen in the path chart to the right, so the DFSM is also its minimized DFSM.
Because any number of a's or b's can be included at the beginning of an accepted string but must end with the sequence 'ababa', the regular expression for L4 is: a*b*ababa

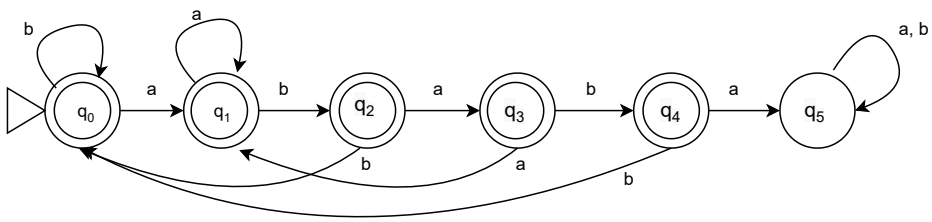
1e. L5 = {w ∈ {a, b}∗ | w does not have ababa as a substring}

NDFSM to test if L5 is regular (This NDFSM is the inverse of L3)



By the pumping theorem, if L5 is regular, then there exists a k >= 1 such that any w in L5 with |w| >=k can be written as w = xyz for some x, y, z in Σ* where $\forall q >= 0$, xy^qz is in L5. For L5, there are 6 states (k) and if y was selected as the first or last state to be repeated, the result would still be an element of L5, without "ababa" as a substring. Because L5 passes the pumping theorem and a proper non-deterministic state automata can be formed to accept L5, L5 is regular.

DFSM

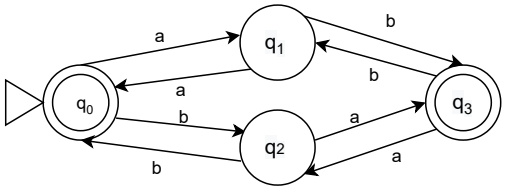


q	a	b
0	1	0
1	1	2
2	3	0
3	1	4
4	5	0
5	5	5

The DFSM for L5 has no states that are equivalent as seen by the path chart on the right so the DFSM is also its minimized DFSM.
Because any combination of a's and b's up to abab can be accepted by L5 and can be followed by or ended by any number of a's or b's, the regular expression for L5 is: a*b*(b + aa + abb + ababb)*(abaa)*(a + ab + aba + abab)*a*b*

1f. L6 = {w ∈ {a, b}∗| #a(w) − #b(w) ≡ 0 (mod 2)}. (#a(w) is the number of a's in w).

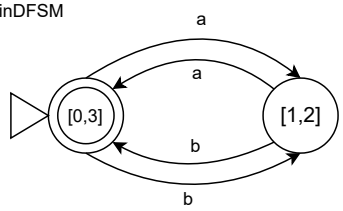
For L6 to be accepted, the number of a's minus the number of b's in the string w would have to make an even number so that when mod 2 is applied the answer is 0. This means that whenever there are both even a's and b's, as well as when there are odd a's and b's, L6 will accept those strings. As such, an accurate DFSM can be formed showing all potential accepting strings and thus L6 is considered regular.



q	a	b
0	1	2
1	0	3
2	3	0
3	2	1

Based on the path chart indicated on the right, the DFSM above has two equivalent states, where states 0 and 3 both go to states 1 and 2 and vice versa. Therefore a minimal DFSM can be formed

minDFSM



Because a and b go to the same state in both directions the regular expression for L6 is: (a+b)*(a+b)*

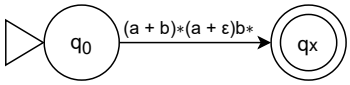
1g. L7 = {w ∈ {a, b}∗| #a(w) − #b(w) = 0}.

By the pumping theorem, if L7 is regular, then there exists a k >= 1 such that any w in L7 with |w| >=k can be written as w = xyz for some x, y, z in Σ* where $\forall$q >= 0, xy^qz is in L7. For L7 if y was selected in any of the first k letters of w and was pumped, it would change the balance of the number of a's and b's in w, meaning that the condition #a(w) − #b(w) = 0 would be false. Because L7 does not pass the pumping theorem, L7 is not regular.

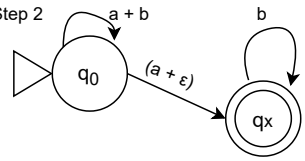
1h. L8 = L((a + b)∗(a + ε)b∗)

L8 is shown as a regular expression which is a condition needed for a regular language. Therefore L8 is regular. Because L8 is written as the language that accepts the regular expression (a + b)∗(a + ε)b∗, it needs to be converted to its corresponding DFSM starting from the simplest step (step 1). Step 2 shows what happens when loops denoted by * in the regular expression are expanded. Step 3 shows the final product after the unions (+) are expanded. The (a+b)* is a loop for a+b off of the initial state q0. To show that a or ε would go towards the final accepting state (a+ε), q1 shows ε going towards it while a and b go to q0 from q1 indicating that it is a union relationship.

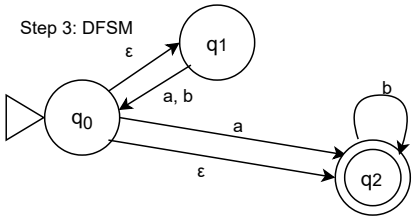
Step 1



Step 2



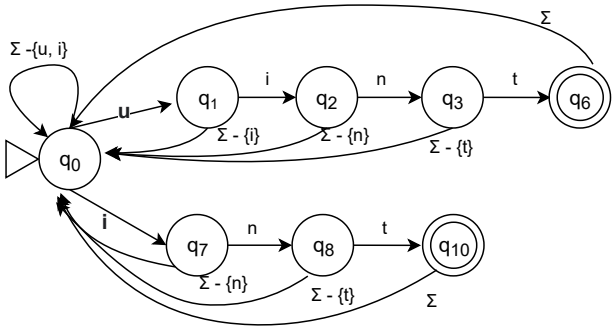
Step 3: DFSM



L8 is shown in regular expression so the regular expression for L8 is (a + b)∗(a + ε)b∗

2a. The minimal DFSM to solve the multi-pattern searching problem for the patterns p1 = int, p2 = uint.
Any other option in the DFSM below would go to a dead state and no other states are equivalent

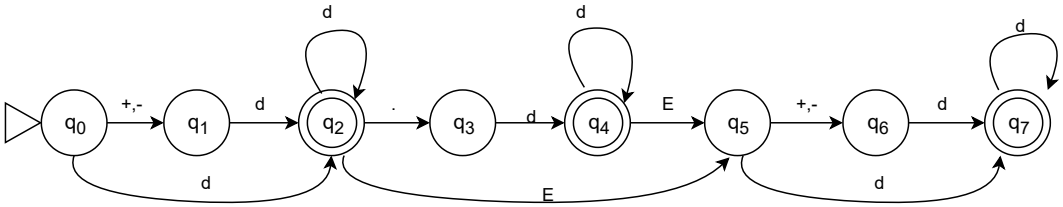
Starting from q0, the algorithm can search for either p1 or p2. If a u is detected by the machine, it searches for an i, an n, and a t, respectively. If any other letter comes up instead, the machine returns to its initial state and starts again. If searching for p1, the machine waits to see if an i was received, then an n, then a t, respectively, where any other letter coming up or coming up out of sequence returns the machine to its initial state.



2b. The language accepted by the DFSM above would be L(Σ*(p1 + p2)) as in L(Σ*(int + uint)). The equivalence of ≈L would contain all possible combinations of the alphabet with p1, and with p2. The equivalence classes would appear as follows:

[ε] includes start state
[int, aint, aaint, uint, auint, aauint, ...] includes all strings in the language
[a, aa, aaa, aab, x, ...] includes all strings not containing 'u' or any letter (also not followed by 'int')
[ux, aux, aaux, aabux, ...] includes all strings that contain 'u' but is not followed by 'int'
[ix, aix, aaix, ux, auix, ...] includes all strings that start with 'i' or 'ui' but does not have 'nt' following it it
[ux, uinx, auinx, aauinx, aabuinx, ...] includes all strings that have 'in' or 'uin' but are not followed by 't'

3. A problem can be considered decidable if its corresponding algorithm to solve it is 1. correct and 2. halts. To show that the algorithm for all floating-point numbers being in L(M) with the possible exception of finitely many ones is decidable, it needs to be proven to halt.

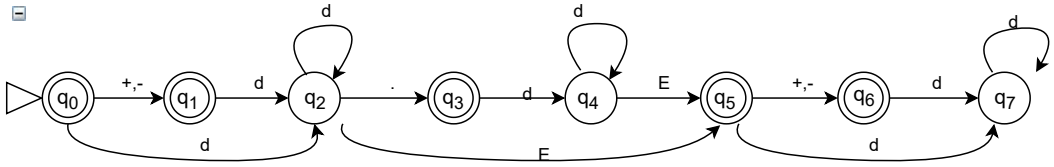


Emptiness

To prove that M is correct it needs to be proven to be not empty. Starting from the initial state (q0) and following any path, at some point an accepting state will be reached for any w put into M. Therefore w is accepted by M proving that L(M) is not empty.

Totality

To prove that M is correct it needs to be proven to have totality ($L(M) = \Sigma_M^*$). To prove that, the above DFSM can be inverted to accept the complement of L(M)



M' above accepts $(L(M))^c$ and because this FSM has an accepting state that is reachable from the start state, M' is not empty and therefore L(M) has totality.

Finiteness

A decision procedure can also correctly answer questions such as if L(M) is finite. Looking at the FSM for M, when starting from the initial state, and following any of the paths, a cycle will be encountered for floating point values. This cycle implies that L(M) is not finite and can have infinitely many accepting values.

Based on the fact that M is shown to be correct based, in accepting correct strings and rejecting incorrect ones and based on the above proofs, as well as the fact that it is shown to terminate (shows that it has strings/floating point numbers that are accepted by it), M can therefore be concluded to be decidable.